

プレイヤーの攻撃

functionを使用して、PlayerにSTATEデザインパターンを導入し、ボタン押下でプレイヤーが攻撃アニメーションを取るようになります。

```
Player.h
```

```
public:
```

```
    // 状態
```

```
    enum class STATE
```

```
    {
```

```
        NONE,
```

```
        PLAY,
```

```
        ATTACK,
```

```
    };
```

```
    // アニメーション種別
```

```
    enum class ANIM_TYPE
```

```
    {
```

```
        IDLE,
```

```
        RUN,
```

```
        FAST_RUN,
```

```
        JUMP,
```

```
        SPINNING_SLASH,
```

```
    };
```

```
private:
```

```
    // 状態管理
```

```
    STATE state_;
```

```
    // 状態管理(状態遷移時初期処理)
```

```
    std::map<STATE, std::function<void(void)>> stateChanges_;
```

```
    // 状態管理(更新ステップ)
```

```
    std::function<void(void)> stateUpdate_;
```

```
    // 状態遷移
```

```
    void ChangeState(STATE state);
```

```
    void ChangeStateNone(void);
```

```
    void ChangeStatePlay(void);
```

```
    void ChangeStateAttack(void);
```

```
// 更新系
void UpdateNone(void);
void UpdatePlay(void);
void UpdateAttack(void);

// 操作
void ProcessMove(void);
void ProcessJump(void);
void ProcessAttack(void);
```

Player.cpp

```
void Player::InitAnimation(void)
{

    ~ 省略 ~

    animationController_>Add(
        static_cast<int>(
            ANIM_TYPE::SPINNING_SLASH), 60.0f, path + "SpinningSlash.mvl");

    // 初期アニメーション
    animationController_>Play(static_cast<int>(ANIM_TYPE::IDLE), true);

}

void Player::InitPost(void)
{

    // 状態管理
    stateChanges_.emplace(
        STATE::NONE, std::bind(&Player::ChangeStateNone, this));
    stateChanges_.emplace(
        STATE::PLAY, std::bind(&Player::ChangeStatePlay, this));
    stateChanges_.emplace(
        STATE::ATTACK, std::bind(&Player::ChangeStateAttack, this));

    // 初期状態
    ChangeState(STATE::PLAY);
```

```

        // 武器の更新
        weapon_>Update();

    }

void Player::ProcessAttack(void)
{

    // ジャンプ中は攻撃できない
    if (isJump_) return;

    auto& ins = InputManager::GetInstance();

    // 攻撃ボタン
    bool isHitKey = ins.IsTrgDown(KEY_INPUT_COLON)
        || ins.IsPadBtnTrgDown(
            InputManager::JOYPAD_N0::PAD1, InputManager::JOYPAD_BTN::LEFT);

    if (isHitKey)
    {
        // スピニングスラッシュアニメーション再生
        animationController_>Play(
            static_cast<int>(ANIM_TYPE::SPINNING_SLASH), false);

        ChangeState(STATE::ATTACK);

    }

}

void Player::ChangeState(STATE state)
{

    // 状態変更
    state_ = state;

    // 各状態遷移の初期処理
    stateChanges_[state_]();

}

```

```

void Player::ChangeStateNone(void)
{
    stateUpdate_ = std::bind(&Player::UpdateNone, this);
}

void Player::ChangeStatePlay(void)
{
    stateUpdate_ = std::bind(&Player::UpdatePlay, this);
}

void Player::ChangeStateAttack(void)
{
    stateUpdate_ = std::bind(&Player::UpdateAttack, this);
}

void Player::UpdateNone(void)
{
}

void Player::UpdatePlay(void)
{
    // 移動操作
    ProcessMove();

    // ジャンプ処理
    ProcessJump();

    // 攻撃処理
    ProcessAttack();
}

void Player::UpdateAttack(void)
{
    // 移動量を徐々に減少
    movePow_ = AsoUtility::Lerp(movePow_, AsoUtility::VECTOR_ZERO, 0.05f);

    // アニメーションの再生終了

```

```
if (animationController->IsEnd())  
{  
    ChangeState (STATE::PLAY);  
}  
  
}
```

追加のアニメーションデータは下記からダウンロード

Teams . . . 07_3DAction/02_Data/SpinningSlash.mvl

プレイヤーが攻撃モーションを取ったらOK。

